NATURAL LANGUAGE PROCESSING

PROJECT 2

CONTENT SUMMARIZATION

Abstractive Summarization using BART
versus
Extractive Summarization using TextRank

STUDENT AND FACULTY LEARNING MANUAL

Beginner-friendly concepts • Complete source code • VS Code execution
Troubleshooting • Experiments • Viva questions • Teaching plan

Prepared from the NLP Projects specification and the project files used in the classroom execution session.

How to use this manual

This manual is designed for a learner who is new to NLP project implementation. Follow the sections in order: understand the task, prepare the environment, run the dashboard, experiment with the outputs, and then read the code. Faculty members can use the teaching plan and assessment rubric near the end.

Part	What it covers
1. Project overview	Problem statement, learning outcomes and workflow
2. Concepts	Summarization, BART, TextRank and beam search
3. Environment	Folder structure, dependencies and terminal commands
4. Application use	Streamlit interface and parameter controls
5. Complete code	Full app.py, summarizer.py and sample article
6. Code walkthrough	Purpose of each important statement and function
7. Troubleshooting	NLTK punkt / punkt_tab issue and common fixes
8. Experiments	Suggested tests, observation sheet and interpretation
9. Assessment	Exercises, viva questions and faculty teaching plan
10. References	Project PDF and software/model resources

1. Project overview

Project title: Content Summarization – Abstractive versus Extractive Summarization.

Objective: Build two summarization pipelines and compare their behaviour on the same article. The first pipeline uses the pretrained facebook/bart-large-cnn sequence-to-sequence model to generate a summary. The second uses the TextRank algorithm through the Sumy library to select important sentences from the original article.

Learning outcome	What the learner should be able to do
Differentiate methods	Explain how abstractive and extractive summaries differ.
Use a pretrained model	Load BART tokenizer and sequence-to-sequence model through Transformers.
Generate text	Understand input tokenization, truncation, beam search and decoding.
Use TextRank	Explain sentence selection and graph-based ranking at a high level.
Build an interface	Run and use a Streamlit dashboard.
Evaluate outputs	Compare coverage, clarity, factual correctness and information loss.

Expected project outcome

A local web application in which a user pastes an article, adjusts summary settings, and generates a BART abstractive summary and a TextRank extractive summary for side-by-side comparison.

2. Core concepts

2.1 What is text summarization?

Text summarization is the process of producing a shorter version of a document while preserving its most important information. The input may be an article, report, research paper or other text. The goal is not simply to delete words: the summary should communicate the main ideas accurately and clearly.

2.2 Abstractive summarization

Abstractive summarization generates a new piece of text that expresses the main ideas of the source. It can combine information from multiple sentences, shorten explanations and use different wording. This project uses BART, a pretrained encoder–decoder model exposed through Hugging Face Transformers.

- The model reads the input article and generates a new sequence of tokens as the summary.
- The output may paraphrase the source instead of copying complete sentences.
- Because it generates language, the output must be checked for unsupported statements, omissions or altered meaning.

2.3 Extractive summarization

Extractive summarization selects important sentences that already exist in the source document. It does not normally rewrite those sentences. This project uses TextRank through Sumy.

- The article is divided into sentences.
- TextRank estimates the importance of sentences using a graph-based ranking process.
- The requested number of high-ranking sentences is returned from the original article.

2.4 BART versus TextRank

Aspect	BART (abstractive)	TextRank (extractive)
Main operation	Generates a new summary sequence	Selects existing source sentences
Approach	Pretrained deep-learning sequence-to-sequence model	Graph-based sentence ranking
Wording	Can paraphrase or combine ideas	Retains the wording of selected sentences
Training	Uses a pretrained model checkpoint	No project-specific model training is required
Strength	Can produce compact, fluent summaries	Makes source wording easy to trace
Caution	May omit or alter facts; verify output	May be less fluent or contain redundant sentences

2.5 What is BART?

BART stands for Bidirectional and Auto-Regressive Transformers. It uses an encoder–decoder Transformer architecture. In this project, the checkpoint name is facebook/bart-large-cnn. The tokenizer converts article text into token IDs that the model can process. The model then generates summary token IDs, and the tokenizer converts those IDs back into readable text.

Component	Role in this project
AutoTokenizer	Converts the input article into model tokens and decodes generated tokens.
AutoModelForSeq2SeqLM	Loads the encoder–decoder model used to generate the summary.
model.generate()	Performs sequence generation using the configured decoding parameters.
tokenizer.decode()	Converts the generated token IDs into a text string.

2.6 What is TextRank?

TextRank is a graph-based ranking method inspired by PageRank. For summarization, sentences are treated as graph nodes. Relationships between sentences are represented by weighted connections based on their similarity. The ranking procedure estimates which sentences are most central or important in the article. Sumy provides the TextRankSummarizer implementation used here.

The project code does not train a neural network for TextRank. It parses the article, tokenizes it with Sumy, applies the TextRank summarizer and returns the selected sentences.

2.7 What is beam search?

Beam search is a decoding strategy used while generating text. Instead of keeping only one possible next-token path at each step, it keeps a small set of promising partial sequences (the beam). The model expands and scores these candidates and continues with the strongest options. In this dashboard, the num_beams control sets the beam width. A larger value may increase generation time; it does not guarantee a better summary.

Important distinction

The BART maximum and minimum settings are token limits, not word counts or sentence counts. TextRank's sentence-count setting asks for a number of source sentences.

3. Project architecture and files

File / folder	Purpose
nlp_projects/	Root workspace used for the NLP projects.
nlp-env/	Shared Python virtual environment. Do not create a separate environment for this project.
content_summarization/app.py	Streamlit user interface and controls; calls the two summary functions.
content_summarization/summarizer.py	Contains the BART and TextRank implementation.
content_summarization/ sample_article.txt	Companion sample text file supplied with the project.

Note about the sample text file

The current app.py defines SAMPLE_ARTICLE directly in the Python source and uses it as the default text-area value. It does not automatically read sample_article.txt. The text file is a separate companion copy that can be opened or pasted into the application.

3.1 Processing workflow

Stage	BART branch	TextRank branch
1. Input	The same article is entered in Streamlit.	The same article is entered in Streamlit.
2. Preparation	Tokenizer converts text to token IDs; input is capped at 1024 tokens.	Sumy parser and English tokenizer split the text into a document/sentences.
3. Summarization	BART generates summary token IDs using beam search.	TextRank ranks source sentences.
4. Output	Generated token IDs are decoded into new text.	Selected original sentences are joined into a summary string.
5. Display	Streamlit displays the result in the left column.	Streamlit displays the result in the right column.

4. Environment setup and terminal execution

The classroom workspace uses one shared environment named nlp-env under the nlp_projects root folder. This avoids reinstalling large packages separately for each project.

4.1 Open the correct folder

In VS Code, open the nlp_projects folder. Choose Terminal → New Terminal. In PowerShell, navigate to the summarization folder:

```
cd C:\Users\spriy\nlp_projects\content_summarization
```

To list the project files, run:

```
dir | Select-Object Name, Length
```

4.2 Activate the existing environment

If the terminal prompt does not begin with (nlp-env), activate the environment. The project folder is one level below the virtual-environment folder, so the relative path begins with ..\.

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
..\nlp-env\Scripts\Activate.ps1
```

The first command temporarily permits PowerShell scripts in the current terminal process only. It does not permanently change the computer-wide or user-wide execution policy. The second command activates the existing virtual environment.

Activation check

The terminal prompt should begin with (nlp-env), for example: (nlp-env) PS C:\Users\spriy\nlp_projects\content_summarization>

4.3 Required Python packages

Package	Purpose	Status in the classroom setup
transformers	Loads BART tokenizer and model.	Already installed.
torch	Runs the neural network model.	Already installed.
streamlit	Builds the browser-based application.	Installed when the check reported it missing.
sumy	Provides TextRank summarization.	Installed when the check reported it missing.
nltk data: punkt, punkt_tab	Tokenizer resources needed by the sentence-tokenization path.	Downloaded after the TextRank tokenizer error.

4.4 Check installed packages

```
python -m pip show streamlit sumy transformers torch
```

If Streamlit and Sumy are missing, install them inside the active nlp-env environment:

```
python -m pip install --default-timeout=1000 --retries 10 --prefer-binary streamlit sumy
```

Transformers and PyTorch were already present in the classroom environment, so they did not need to be reinstalled.

4.5 Download NLTK tokenizer data

```
python -m nltk.downloader punkt punkt_tab
```

This downloads language data used by tokenization. Installing the Python package and downloading its data resources are separate operations.

4.6 Launch the Streamlit application

```
streamlit run app.py
```

Streamlit starts a local web server. If the browser does not open automatically, navigate to:

```
http://localhost:8501
```

Keep the terminal window open while using the application. To stop the local Streamlit server, return to the terminal and press Ctrl+C.

First model load

The first BART run downloads the pretrained model files and may take longer, depending on internet speed. Later runs normally reuse the local Hugging Face cache. The model can run on a CUDA GPU if one is available; otherwise the code selects the CPU.

5. Using the Streamlit application

5.1 Main screen

The application title is “Content Summarizer: Abstractive vs Extractive”. It contains an article text area, four controls and one button to generate both summaries.

Interface item	Default / range	Meaning
Article text	Preloaded with a sample AI article.	Text to be summarized. You can replace it with your own article.
BART maximum summary tokens	40–200; default 150; step 10.	Upper limit supplied to BART generation, subject to safeguards for short inputs.
BART minimum summary tokens	5–100; default 50; step 5.	Lower limit supplied to BART generation, subject to safeguards for short inputs.
Beam search (num_beams)	2, 4, 6 or 8; default 4.	Number of candidate beams retained during generation.
TextRank source sentences	1–8; default 3.	Number of original sentences requested from TextRank.
Generate both summaries	Button.	Calls the BART and TextRank summary functions.

5.2 Suggested first execution

1. Keep the preloaded article in the Article text area.
2. Keep the default settings: BART maximum 150, minimum 50, num_beams 4 and TextRank count set to three.
3. Click Generate both summaries.
4. Wait for BART generation to finish. The first run may take longer because the model is loaded and cached.
5. Read the BART output on the left and TextRank output on the right.
6. Compare wording, coverage, readability and whether the summary preserves the article’s main points.

5.3 Understanding the displayed summaries

The BART result is generated text and may use wording not present in the article. The TextRank result is assembled from selected source sentences. Do not assume either result is automatically perfect: a person should check factual coverage and meaning.

6. Complete source code: app.py

The following is the complete app.py used in the supplied classroom project package. It defines the Streamlit page, sample article, controls, button behaviour, error handling and two-column result display.

app.py — complete file (interface, controls and execution)

```
"""Streamlit interface comparing BART and TextRank summaries.

Run from this folder with:
    streamlit run app.py
"""

import streamlit as st
from summarizer import abstractive_summary, extractive_summary

st.set_page_config(page_title="NLP Content Summarizer", page_icon="📝", layout="wide")
st.title("Content Summarizer: Abstractive vs Extractive")
st.write(
    "Paste an article and compare a generated BART summary with sentences "
    "selected by the TextRank algorithm."
)

SAMPLE_ARTICLE = """Artificial intelligence is changing how organizations analyze information and make decisions. In
healthcare, machine-learning systems can help clinicians review medical images, identify patterns in patient
records, and prioritize cases that may need attention. These systems are intended to support professional judgement
rather than replace doctors.

In education, intelligent learning platforms can adapt exercises to a learner's progress. They can identify topics
that require additional practice and provide feedback quickly. However, teachers remain important because they
understand the social and individual needs of students.

Businesses also use natural language processing to process customer messages, summarize reports, classify documents,
and search large collections of text. Such tools can reduce repetitive work, but their outputs should be checked for
accuracy, privacy, and bias. The quality of an AI system depends on its data, evaluation, and the way people use its
results.

Researchers are therefore studying explainability, fairness, and responsible deployment. A successful AI application
combines technical performance with human oversight, clear objectives, and continuous evaluation."""

article = st.text_area(
    "Article text",
    value=SAMPLE_ARTICLE,
    height=260,
    help="The BART implementation truncates input to a maximum of 1024 tokens, as specified in the project guide.",
)

col_a, col_b = st.columns(2)
with col_a:
    max_length = st.slider("BART maximum summary tokens", 40, 200, 150, 10)
    min_length = st.slider("BART minimum summary tokens", 5, 100, 50, 5)
with col_b:
    num_beams = st.selectbox("Beam search (num_beams)", [2, 4, 6, 8], index=1)
    sentence_count = st.slider("TextRank: number of source sentences", 1, 8, 3)

if st.button("Generate both summaries", type="primary", disabled=not article.strip()):
    with st.spinner("Generating BART summary. The first run downloads the model and may take some time..."):
        try:
            bart_result = abstractive_summary(
                article, max_length=max_length, min_length=min_length,
                num_beams=num_beams,
            )
            st.session_state["bart_result"] = bart_result
        except Exception as exc:
            st.error(f"BART summarization failed: {exc}")
            st.info("Check the internet connection on the first run and confirm that PyTorch and Transformers are
installed.")

    try:
        textrank_result = extractive_summary(article, sentences_count=sentence_count)
        st.session_state["textrank_result"] = textrank_result
    except LookupError as exc:
        st.error(f"TextRank tokenizer data is missing: {exc}")
        st.code("python -m nltk.downloader punkt punkt_tab", language="powershell")
    except Exception as exc:
        st.error(f"TextRank summarization failed: {exc}")
```

app.py — continuation (displaying the two summaries)

```
if "bart_result" in st.session_state or "textrank_result" in st.session_state:
    st.divider()
    left, right = st.columns(2)
    with left:
        st.subheader("1. Abstractive summary (BART)")
        st.caption("BART generates a new sequence of words; it may paraphrase the article.")
        st.write(st.session_state.get("bart_result", "No BART result available."))
    with right:
        st.subheader("2. Extractive summary (TextRank)")
        st.caption("TextRank selects important sentences from the original article.")
        st.write(st.session_state.get("textrank_result", "No TextRank result available."))

st.divider()
st.caption("Classroom discussion: compare factual coverage, wording, readability, and whether each summary
introduces or omits information.")
```

6.1 app.py code walkthrough

Code element	Explanation
import streamlit as st	Imports Streamlit and gives it the short name st.
from summarizer import ...	Imports the two functions implemented in summarizer.py.
st.set_page_config(...)	Sets browser tab title, icon and wide page layout.
st.title() / st.write()	Displays the application heading and introduction.
SAMPLE_ARTICLE	Stores the default article as a multiline Python string.
st.text_area()	Creates the editable article input box.
st.columns(2)	Creates two side-by-side areas for settings and later results.
st.slider() / st.selectbox()	Collects the summary parameters from the user.
st.button()	Starts processing when the user clicks Generate both summaries.
st.spinner()	Shows a progress message while BART is generating.
try / except	Catches model or tokenizer errors and shows helpful messages instead of silently stopping.
st.session_state	Stores results so they remain available across Streamlit reruns.
st.divider(), st.subheader(), st.caption()	Organizes and labels the two result panels.

6.2 Why session_state is used

Streamlit reruns a script after many user interactions. st.session_state stores values across those reruns during the active session. The application stores BART output in bart_result and TextRank output in textrank_result, then checks whether either result exists before displaying the result section.

6.3 Why there are separate try/except blocks

BART and TextRank are executed independently. If one method fails, the app can display an error for that method while still retaining a result from the other method. A LookupError from TextRank is handled specially because it commonly indicates missing NLTK tokenizer data.

7. Complete source code: summarizer.py

This file contains the actual summarization logic. app.py collects user input and calls these functions; summarizer.py performs model loading, tokenization, generation and sentence selection.

summarizer.py — BART model loading and abstractive generation

```
"""Abstractive BART and extractive TextRank summarization utilities."""

from functools import lru_cache

MODEL_NAME = "facebook/bart-large-cnn"


@lru_cache(maxsize=1)
def load_bart_model():
    """Load and cache BART so Streamlit does not reload it on every rerun."""
    import torch
    from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
    model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)
    model.eval()
    return tokenizer, model, device


def abstractive_summary(article, max_length=150, min_length=50, num_beams=4):
    """Generate a new summary with BART. Inputs are truncated at 1024 tokens."""
    if not article or not article.strip():
        raise ValueError("Please provide article text.")

    import torch

    tokenizer, model, device = load_bart_model()
    encoded = tokenizer(
        article.strip(),
        return_tensors="pt",
        max_length=1024,
        truncation=True,
    )
    input_token_count = int(encoded["input_ids"].shape[1])
    encoded = {key: value.to(device) for key, value in encoded.items()}

    # Keep the requested PDF defaults for normal-length articles, while
    # reducing limits for short inputs so generation does not demand more
    # summary tokens than the source can reasonably support.
    safe_max = min(max_length, max(20, input_token_count - 2))
    safe_min = min(min_length, max(5, safe_max // 2))
    if safe_min >= safe_max:
        safe_min = max(1, safe_max - 1)

    with torch.no_grad():
        summary_ids = model.generate(
            **encoded,
            max_length=safe_max,
            min_length=safe_min,
            do_sample=False,
            num_beams=num_beams,
            early_stopping=True,
        )
    return tokenizer.decode(summary_ids[0], skip_special_tokens=True)
```

summarizer.py — continuation (TextRank extraction)

```
def extractive_summary(article, sentences_count=3):
    """Select important original sentences using Sumy's TextRank."""
    if not article or not article.strip():
        raise ValueError("Please provide article text.")

    from sumy.parsers.plaintext import PlaintextParser
    from sumy.nlp.tokenizers import Tokenizer
    from sumy.summarizers.text_rank import TextRankSummarizer

    parser = PlaintextParser.from_string(article.strip(), Tokenizer("english"))
    summarizer = TextRankSummarizer()
    selected_sentences = summarizer(parser.document, sentences_count=sentences_count)
    return " ".join(str(sentence) for sentence in selected_sentences)
```

7.1 BART function walkthrough

Statement / operation	Explanation
MODEL_NAME = "facebook/bart-large-cnn"	Names the pretrained summarization checkpoint.
@lru_cache(maxsize=1)	Caches the loaded model and tokenizer so they are not loaded repeatedly during Streamlit reruns.
AutoTokenizer.from_pretrained(...)	Loads the tokenizer files associated with the BART checkpoint.
AutoModelForSeq2SeqLM.from_pretrained(...)	Loads the sequence-to-sequence model weights and configuration.
torch.cuda.is_available()	Checks whether a CUDA-capable GPU is available to PyTorch.
model.to(device)	Moves the model to GPU or CPU, depending on availability.
model.eval()	Places the model in evaluation mode.
tokenizer(..., max_length=1024, truncation=True)	Converts the article into model inputs and truncates input beyond 1024 tokens.
torch.no_grad()	Disables gradient tracking for inference, reducing memory and computation overhead.
model.generate(...)	Generates output tokens using selected length limits and beam-search settings.
tokenizer.decode(..., skip_special_tokens=True)	Converts the generated token IDs into readable text and omits special tokens.

7.2 Why the code uses safe_max and safe_min

A very short article may contain fewer tokens than the requested maximum and minimum summary lengths. The code therefore adjusts the requested generation limits using input_token_count, safe_max and safe_min. This is an implementation safeguard added to avoid asking the model for an unsuitable summary length on short input. For ordinary longer articles, the UI defaults follow the project specification: maximum 150, minimum 50 and four beams.

7.3 TextRank function walkthrough

Statement	Explanation
PlaintextParser.from_string(...)	Creates a Sumy document from the article text.
Tokenizer("english")	Uses Sumy's English tokenizer to process the text.
TextRankSummarizer()	Creates the TextRank summarizer.
summarizer(parser.document, sentences_count=...)	Ranks sentences and returns the requested number.
" ".join(str(sentence) ...)	Joins selected sentences into one displayable string.

8. Supporting sample article

The folder also contains sample_article.txt. The following is its complete content. The application currently embeds the same sample topic directly in app.py, rather than reading this file automatically.

sample_article.txt — complete content

```
Artificial intelligence is changing how organizations analyze information and make decisions. In
healthcare, machine-learning systems can help clinicians review medical images, identify patterns in
patient records, and prioritize cases that may need attention. These systems are intended to support
professional judgement rather than replace doctors.

In education, intelligent learning platforms can adapt exercises to a learner's progress. They can
identify topics that require additional practice and provide feedback quickly. However, teachers
remain important because they understand the social and individual needs of students.

Businesses also use natural language processing to process customer messages, summarize reports,
classify documents, and search large collections of text. Such tools can reduce repetitive work, but
their outputs should be checked for accuracy, privacy, and bias. The quality of an AI system depends
on its data, evaluation, and the way people use its results.

Researchers are therefore studying explainability, fairness, and responsible deployment. A successful
AI application combines technical performance with human oversight, clear objectives, and continuous
evaluation.
```

8.1 Using another article

To test a new article, click inside the Article text field in the browser, select the existing text, paste your own article and click Generate both summaries. For a fair comparison, use the same article for both methods.

> **Input length reminder**
>
> BART tokenization truncates the article at a maximum of 1024 tokens in this implementation. If an article is longer, the later portion may not be included in the BART input. TextRank does not use that BART tokenization step.

9. Troubleshooting and the classroom execution log

9.1 Actual issue encountered: NLTK punkt_tab missing

After the Streamlit application launched and BART loaded, the first attempt to generate both summaries displayed a TextRank tokenizer error. The message indicated that NLTK tokenizer data was missing and named the resource punkt_tab. This was not a missing Sumy installation: the Python library was installed, but its separate language data was absent.

> **Error message in plain language**
>
> TextRank asked the tokenizer to split the English article into sentences. The required NLTK tokenizer files were not present in the searched NLTK data folders, so the TextRank branch could not continue.

Correction: stop Streamlit with Ctrl+C, make sure (nlp-env) is active, and run:

```
python -m nltk.downloader punkt punkt_tab
```

After the download finishes, restart the application:

```
streamlit run app.py
```

Open or refresh http://localhost:8501 and click Generate both summaries again.

9.2 Other common issues

Symptom	Likely cause	Action
ModuleNotFoundError: streamlit	Streamlit is not installed in the active environment.	Activate nlp-env; install streamlit with python -m pip.
ModuleNotFoundError: sumy	Sumy is not installed in the active environment.	Activate nlp-env; install sumy.
TextRank tokenizer LookupError	NLTK language data is missing.	Run python -m nltk.downloader punkt punkt_tab.
BART first run takes a long time	Model files are downloading or model is loading on CPU.	Keep the terminal open; check internet access; allow the first download to finish.
Browser does not open	Streamlit server is running but browser launch did not occur.	Open http://localhost:8501 manually.
Port 8501 is already in use	Another Streamlit process may still be running.	Stop the earlier terminal with Ctrl+C or use the port reported by Streamlit.
BART output seems incomplete	The input may exceed the 1024-token truncation limit.	Use a shorter article or split it into sections.
A warning appears during model loading	Some warnings are informational and do not stop execution.	Check whether the model completes loading and the app produces a result; investigate only if a traceback or failed output occurs.

9.3 Classroom execution log

Step completed in the session	Observed status
Opened content_summarization in VS Code.	Project files app.py, summarizer.py and sample_article.txt were visible.
Activated shared nlp-env.	Terminal prompt showed (nlp-env).
Checked packages.	Transformers and PyTorch were present; Streamlit and Sumy were missing.
Installed Streamlit and Sumy.	Package checks confirmed installation.
Ran streamlit run app.py.	Streamlit opened the application at localhost:8501.
Loaded BART checkpoint.	Model weights reached 100% in the terminal.
First generation attempt.	BART output was available; TextRank identified missing NLTK punkt_tab data.
Downloaded tokenizer resources and tested again.	User confirmed the issue was resolved and requested the learning notes.

10. Experiments and output analysis

10.1 Recommended experiments

Experiment	Change to make	What to observe
A. Baseline	Use defaults: max 150, min 50, beams 4, TextRank 3 sentences.	Record both outputs and note the main ideas retained.
B. TextRank length	Change sentence count from 3 to 1, then 5.	Observe how the extractive summary changes in length and coverage.
C. Beam width	Compare num_beams 2, 4, 6 and 8.	Observe differences in generated wording, processing time and factual coverage.
D. BART maximum length	Change maximum from 100 to 180 while keeping a suitable minimum.	Observe how much detail appears in the generated summary.
E. Short input	Use a two- or three-sentence article.	Observe how the code adjusts generation limits to suit short input.
F. Long input	Paste an article longer than the 1024-token BART input limit.	Discuss truncation: BART receives a limited input sequence.
G. Different topic	Try a scientific, education or technology article.	Check whether important domain terms and facts remain accurate.

10.2 Student observation sheet

Observation item	BART output	TextRank output
Summary length (words)	_____	_____
Main idea retained?	Yes / Partly / No	Yes / Partly / No
Uses source sentences exactly?	Yes / No / Partly	Yes / No / Partly
Readability (1–5)	____	____
Factual accuracy (1–5)	____	____
Important omission noticed	_____	_____
One strength	_____	_____
One limitation	_____	_____

10.3 How to compare summaries fairly

- Use the same source article for both methods.
- Check whether the key entities, dates, quantities and conclusions are preserved.
- Do not judge only by shorter length; check whether important context was lost.
- For BART, verify that generated wording has not introduced a claim that the article did not support.
- For TextRank, check whether the selected sentences make sense when read together.
- Record the chosen settings so another student can reproduce the experiment.

10.4 Optional evaluation ideas

For an introductory classroom lab, qualitative comparison is sufficient. As an extension, students may compare generated summaries with human-written reference summaries using ROUGE metrics, or ask human reviewers to score factual consistency, readability and coverage. Automated overlap scores should not be treated as proof that a summary is factually correct.

11. Student exercises

1. Explain the difference between abstractive and extractive summarization using an example of your own.
2. Replace the sample article with a paragraph from a news report or textbook chapter. Generate both summaries and compare them.
3. Change the TextRank sentence count from 3 to 5. Record which additional sentences are included.
4. Change num_beams from 4 to 8. Observe whether the summary wording or generation time changes.
5. Use a short article and explain why the summarizer code adjusts the requested minimum and maximum lengths.
6. Explain the purpose of punkt and punkt_tab in the error encountered during execution.
7. Identify the statements that load the BART model only once and explain why caching is useful.

12. Viva questions with answer guidance

1. What is text summarization?

Answer guidance: Creating a shorter version of a document while retaining its important information.

2. What is abstractive summarization?

Answer guidance: Generating new wording to express the main ideas of the source.

3. What is extractive summarization?

Answer guidance: Selecting important sentences or text spans from the original document.

4. Which model is used for abstractive summarization here?

Answer guidance: facebook/bart-large-cnn.

5. What is BART?

Answer guidance: A denoising sequence-to-sequence Transformer with encoder and decoder components.

6. Which library loads BART?

Answer guidance: Hugging Face Transformers.

7. What is the role of a tokenizer?

Answer guidance: Converts text into token IDs for the model and decodes output token IDs.

8. What is beam search?

Answer guidance: A decoding strategy that retains multiple promising partial output sequences.

9. What does num_beams control?

Answer guidance: The beam width, or number of candidate sequences kept during generation.

10. Which library provides TextRank in this project?

Answer guidance: Sumy.

11. Does TextRank generate entirely new sentences?

Answer guidance: No. It selects sentences from the original document.

12. What is punkt_tab?

Answer guidance: An NLTK tokenizer data resource required by the tokenization path used in the environment.

13. Why was the NLTK downloader needed?

Answer guidance: The Sumy package was installed, but the separate NLTK tokenizer data had not been downloaded.

14. What does lru_cache do here?

Answer guidance: Caches the loaded BART tokenizer, model and device so repeated calls do not reload them.

15. Why is model.eval() used?

Answer guidance: To place the neural network in evaluation/inference mode.

16. Why is torch.no_grad() used?

Answer guidance: To disable gradient calculations during inference.

17. What is the maximum BART input length in this implementation?

Answer guidance: 1024 tokens, with truncation enabled.

18. What does Streamlit do?

Answer guidance: Provides a simple interactive web interface for the summarization functions.

19. Why use session_state?

Answer guidance: To preserve generated results across Streamlit reruns within the session.

20. Which summarizer is always better?

Answer guidance: Neither. The choice depends on the task; both outputs must be evaluated for coverage and factual reliability.

13. Faculty teaching plan and assessment

Teaching block	Suggested duration	Faculty activity / expected learner outcome
Concept introduction	15 minutes	Explain summarization, abstractive and extractive approaches; use one short paragraph as an example.
Model and algorithm	20 minutes	Introduce BART encoder–decoder generation, tokenization, beam search and TextRank sentence ranking.
Environment verification	10 minutes	Check nlp-env, package installation and NLTK tokenizer resources.
Application demonstration	15 minutes	Run Streamlit and explain the article input and controls.

Teaching block	Suggested duration	Faculty activity / expected learner outcome
Hands-on experiments	30 minutes	Students change article text and parameters, generate outputs and fill the observation sheet.
Discussion and assessment	20 minutes	Compare accuracy, coverage, readability and limitations; conduct viva or short quiz.

13.1 Suggested assessment rubric

Criterion	Marks	Evidence
Environment and execution	10	Correctly activates environment and runs the Streamlit app.
Conceptual understanding	20	Explains abstractive and extractive summarization, BART and TextRank.
Code understanding	25	Explains app.py, summarizer.py, tokenization and generation.
Experimentation	20	Runs at least three controlled tests and records parameter values.
Result analysis	15	Compares coverage, readability, factual consistency and limitations.
Viva / troubleshooting	10	Explains and resolves the NLTK tokenizer data issue.
Total	100	

13.2 Faculty discussion prompts

- Can an abstractive summary sound fluent but still be factually incorrect? Why?
- Why might an extractive summary be less coherent when its selected sentences are read together?
- What is the effect of increasing the number of TextRank sentences?
- Why should the same source article be used to compare the two methods?
- How does a missing language-data resource differ from a missing Python package?

14. Implementation notes and limitations

- BART input limit: app.py help text and summarizer.py both specify a maximum of 1024 input tokens with truncation. Long articles may be cut off for BART.
- First-run download: the BART model must be downloaded and loaded before generation; network access is needed for the first download unless the files are already cached.
- Hardware: CPU execution is supported but can be slower than GPU execution.
- Generated summaries require review: BART may paraphrase, omit or alter details. Check names, dates, numerical values and conclusions.
- Extractive output can be incomplete or less fluent: TextRank selects source sentences but does not rewrite transitions between them.
- The sample_article.txt file is not loaded dynamically by current app.py; the default article is embedded in the Streamlit source file.
- This is a teaching prototype and a comparison tool, not a production document summarization service.

15. References and source alignment

Primary course source: NLP Projects PDF supplied for this project, Project 2: Content Summarizer (Abstractive vs. Extractive Summarization), including the project description and step-by-step implementation. The specification calls for facebook/bart-large-cnn, input truncation up to 1024 tokens, beam-search generation with four beams, and Sumy TextRank selecting three sentences as the baseline.

Environment source: NLP Environment Setup & Duplicate Question Detector Guide (Revised), supplied earlier in this conversation. The shared nlp-env arrangement and PowerShell setup approach are reused for this project.

Implementation source: content_summarization/app.py, content_summarization/summarizer.py and content_summarization/sample_article.txt from the supplied NLP project package. The full app.py and summarizer.py are reproduced in Sections 6 and 7.

15.1 Software and documentation links

Use these official or project-maintained resources when reviewing package behaviour, model details or installation guidance.

Software / model	Official or project resource
Hugging Face Transformers	https://huggingface.co/docs/transformers
BART checkpoint	https://huggingface.co/facebook/bart-large-cnn
PyTorch	https://pytorch.org/docs/
Streamlit	https://docs.streamlit.io/
Sumy	https://github.com/miso-belica/sumy
NLTK data	https://www.nltk.org/data.html

Completion checklist

Before distributing this manual, confirm that students can activate nlp-env, install the project dependencies, download punkt and punkt_tab, launch Streamlit, generate both summaries, explain the main code functions, and evaluate output quality.

16. Student project completion record

Students may complete this record after the practical session. Faculty can use it as a simple verification sheet.

Verification item	Completed (Yes / No)	Date / remarks
Activated the shared nlp-env environment	_____	_____
Verified Streamlit, Sumy, Transformers and PyTorch	_____	_____
Downloaded punkt and punkt_tab tokenizer resources	_____	_____
Launched the Streamlit application	_____	_____
Generated both BART and TextRank summaries	_____	_____
Completed at least three experiments	_____	_____

17. Reflection and improvement

After completing the project, write brief answers to the following questions:

- Which summarization method was more useful for the article you tested, and why?
- What factual or contextual detail did either summary omit or change?
- Which parameter had the most noticeable effect on the output?
- What would you improve before using this application with long research papers or reports?

Learner notes: _____

_____

Faculty comments: _____